

MODULE 3 · WEEK 3

GenAI & LLM-Assisted ETL

Use a large language model as the 'transform' step — pulling structured data out of messy text on Vertex AI / GCP.

Lectures 27 · 29–30 · 32–33 · 35 · 37–38



This week

- 1 **GenAI on GCP** — calling an LLM from Vertex AI
- 2 **The extractor-LLM-POC** — an LLM as the transform step
- 3 **Materializing output** — many JSONL files into one CSV
- 4 **Enhancing the POC** — predicting new fields (e.g. transmission)
- 5 **Trade-offs** — cost, latency, reliability



Why GenAI changes ETL

- Tasks that needed complex regex, custom parsers, or manual labeling can now be done in plain English.
- Describe what you want, hand the model the text, and let it return structured data.
- LLM-assisted ETL is already in production — pulling structure from unstructured documents.

Key idea With an LLM, the 'transform' in ETL can be a prompt instead of a parser.

GenAI on Vertex AI / GCP

- Vertex AI lets you call hosted generative models from your GCP project.
- Authenticate through the project; keep API keys in environment variables, never in code.
- Same cloud project from last week — now with a model endpoint.

```
# keys come from the environment, never hardcoded
import os
API_KEY = os.environ['GENAI_API_KEY']
```

The extractor-LLM-POC

- A proof-of-concept that feeds raw listing text to an LLM and asks for structured JSON back.
- The model returns fields (price, year, make, model, mileage...) as a clean object.
- Each record becomes a small `.jsonl` file — one JSON object per line.

```
prompt = f"Extract price, year, make, model as JSON:\n{text}"  
result = model.generate(prompt)    # -> {"price": 14995, ...}
```

Materializing the LLM output

- You end up with many JSONL files — one per record. That's awkward to model on.
- **Materialize** them: read every JSONL and concatenate into one tidy CSV.
- Add a column that keeps the raw JSONL so you can always trace a row back.

```
import pandas as pd, glob
rows = [pd.read_json(f, lines=True) for f in glob.glob('out/*.jsonl')]
df = pd.concat(rows, ignore_index=True)
df.to_csv('extracted.csv', index=False)
```

Key idea Collapse the scattered JSONL files into one CSV — now it's a normal modeling dataset.



Enhancing the POC

- Add a new target: ask the LLM to also pull **transmission** (easy to forget at the end!).
- Three ways to push further: richer prompts, more fields, and validating the model's output.
- Compare the LLM-extracted fields against your RegEx pipeline from last week.



The trade-offs

- **Cost** — every call is money; batch and cache where you can.
- **Latency** — an API call is far slower than a regex.
- **Reliability** — validate outputs; LLMs can hallucinate or drift from the format.



Lab this week

- Use an LLM API to extract structured data from unstructured text, with the 'transform' step being the model call.
- Handle API keys securely via environment variables, and weigh cost/latency/reliability vs. traditional parsing.



Key takeaways

- An LLM can be the 'transform' step of ETL — structure from plain text via a prompt.
- Run it on Vertex AI / GCP; keep keys in environment variables.
- Materialize many JSONL outputs into one CSV for modeling, keeping a trace column.
- Extend the POC to new fields, and always validate the output.
- Mind the trade-offs: cost, latency, and reliability vs. regex.